IceFEM: A finite element code to simulate fracture and viscous/plastic deformation in ice.

Tim Hageman*

Department of Engineering Science, University of Oxford, Oxford OX1 3PJ, UK

Abstract

Documentation that accompanies the *C++* code IceFEM, available from here. This documentation explains how to compile the code, the usage of the implemented finite element framework, and highlight the main files. The code allows the simulation of the detailed failure behaviour of ice, considering brittle fractures through the phase-field model, capturing shear bands through von Mises plasticity, and capturing creep through Glens law. Several example files are also included, demonstrating how to set up the simulation of a tri-axial compression sample, and of an ice-cliff failure study. If using (part of) this code, please cite T Hageman. *Phase-field fracture modelling of ice: From triaxial tests to ice-cliff collapse*. Computer Methods in Applied Mechanics and Engineering [**?**].

Keywords: Glaciology, Finite elements, Phase-field fracture, Plasticity, Ice mechanics

Contents

1 Introduction **1**
1.1 Installation . . . 1
1.2 Basic usage . . . 1
1.3 Input Files . . . 1

2 Physics Models **6**
2.1 Cosserat phase field model . . . 7
2.2 Thermal model . . . 7
2.3 Variable porosity model . . . 7
2.4 Boundary constraints model . . . 8
2.5 Tri-Axial compression forcing model . . . 8
2.6 Weak external forcing model . . . 9

3 Examples **9**
3.1 Mesh Generation . . . 9
3.2 Results while solving . . . 9
3.3 Post-processing . . . 9

4 References **12**

*Corresponding author
Email address: `tim.hageman@eng.ox.ac.uk` (Tim Hageman)

1. Introduction

1.1. Installation

1.1.1. Required external libraries

The simulation code has been developed to work directly in ubuntu, or within windows using WSL [1]. A script is included which installs all required libraries, *Install_ICEFEM.sh*. While it can be directly run to install and compile all libraries at once, it is strongly recommended that the contents is run line-by-line by copying it into the terminal. This allows for easier debugging in case of errors. The following libraries are required:
Intel OneMKL [2] is used for its linear matrix solvers (and used by PETSc).
Eigen [3] is used for small matrix and vector mutliplications.
RapidJSON [4] is used for reading the input files.
VTK [5] is used for visualisation during simulations.
Highfive [6] is used to write output files in the HDF5 format.
PETSc [7] is used for its implementation of parrallel (MPI) matrices and vectors (and their associated solvers).
igaFEM [8, 9] is used for the mesh generation routines.

Most of the above libraries are easy to install. However, for PETSc care needs to be taken to enable support for C++, link to the Intel MKL library, and enable support for HDF5. This is enabled through the command:

Install_ICEFEM.sh

```
PETSC_ARCH=optimised
./configure PETSC_ARCH=optimised --with-scalapack-include=$MKLROOT/include --with-scalapack-lib="-L$MKLROOT/lib/intel64 -lmkl_scalapack_lp64 -lmkl_blacs_intelmpi_lp64" --with-mpi-dir=$I_MPI_ROOT --with-mkl=$MKLROOT --with-blas-lapack-dir=$MKLROOT --with-mkl_pardiso-dir=$MKLROOT --with-mkl_cpardiso-dir=$MKLROOT --download-hdf5 --download-mumps --with-clanguage=cxx --with-debugging=0 --COPTFLAGS='-O3' --FOPTFLAGS='-O3' --CXXOPTFLAGS='-O3'
```

In addition to the environmental variables set by OneMKL and PETSc ($PETSC_DIR, $ PETSC_ARCH, $MKLROOT), an additional environmental variable is used in the cmake files to locate the other libraries, $LIBRARY_DIR, which should point to the parent folder that contains the eigen, rapidjson, and Highfive folders.

1.1.2. Compiling

The code makes uses of cmake to create appropriate makefiles. To compile the code, first run cmake using *cmake .* from the base directory, after which the code itself can be compiled using *make* (or *make -j 8* to compile in parrallel).

1.2. Basic usage

Once compiled, the code can be run using *./IceCode <inputfile>* to run on a single core, or *mpirun -n <number of cores> ./IceCode <inputfile>* to run in parallel. Example input files are included within the folder *TestCases*. Running the code using the example file will print the progress of the simulation to the terminal, and open several plots showing the state of the system. The code will also save the results to a file, which can be visualised using the *PlotData.m* matlab script available in the test cases' folder.

1.3. Input Files

Example input files are provided in the folder *TestCases*, which also includes the required scripts for mesh generation. The input files are in JSON format, containing the following sections:

1.3.1. General settings

Here, the general options, such as whether backup files are generated, and the amount of information printed during simulations are defined:

TestCases/TriAxialCompression/TriAxial.json

```
    "Restart":{
        "Restartable": false,
        "Saveloc": "Results/Backup",
        "SaveFrequency": 100,
        "BackUps": 3
    },
    "Logs":{
        "InfoLevel": 3
    },
```

with *InfoLevel* defining whtehr just simulation progress is printed(1), information within each time step is printed (2), or detailed information about matrix assembly steps are also included (3).

1.3.2. Mesh settings

Mesh generation is done through external scripts, with the path to the mesh file provided in the input file as:

TestCases/TriAxialCompression/TriAxial.json

```
    "mesh":{
        "dim":3,
        "file":"TestCases/TriAxialCompression/Meshgen/Mesh_5mm_p2_c56.h5",
        "ipcount1D": 3
    },
```

This also defines the dimension of the problem (2D or 3D), and the number of integration points used per dimension (e.g. 2 for 2x2 integration points in 2D, and 2 for 2x2x2 integration points in 3D). Note that the mesh generation also pre-calculates the distribution of the elements across the cores, used for parrallelisation, and hence the meshes are specific to the number of cores used.

1.3.3. Material settings

Next, material settings are defined based on the physics they are used within. Starting with the phase-field settings:

TestCases/TriAxialCompression/TriAxial.json

```
    "properties":{
        "PhaseField":{
            "DamFunction": "Quadratic",
            "DensityDamage": "Step2",
            "GravityDamage": "Step2",
            "ViscoDriving": "None",
            "ViscoDrivingFactor": 0.0,
            "ConsistentEnergy": false,
            "ConsistentDegradation": false,
            "DistributionFunction": "HO_Linear2",
            "k0": 1.0e-12
        },
```

In order of occurance defining the damage functions used for the elastic energy ("Quadratic" or "Cubic"), the degradation functions used for density and gravity (same as for the elastic energy, or "Step2", "Step5", "Step9", or "None"), the degradation function used to define the viscous and plastic driving forces, whether the same phase-field split used for the elastic energy within the phase-field equation is consistently used for the momentum balance as well, the phase-field distribution used ("Linear", "Quadratic", "HO_Linear", or "HO_Quadratic"), and the residual stiffness once the material is fully damaged (i.e. the stiffness of the material once the phase-field variable is zero).

Temperature dependent properties are defined as:

TestCases/TriAxialCompression/TriAxial.json

```
        "Ice":{
            "TemperatureDependent":{
                "TemperatureDependent": true,
                "TRef": 273.15,
                "TOffset": 273.15,
                "QCreep": 150.0e3,
                "ft0": 0.2e6,
                "ftdeg":4.0e4,
                "sydeg": 0.5e6
            },
```

which defines the reference temperature at which the properties are defined, the energy related to viscous creep, the tensile strength at the reference temperature and how much this changes with temperature, and how much the yield strength changes with temperature.

Next, the mechanical properties are defined as:

TestCases/TriAxialCompression/TriAxial.json

```
            "SolidProperties":{
                "Rheology": "DamViscoElasticPlastic",
                "Density": 910,
                "Young": 5.0e9,
                "Poisson": 0.35,
                "G_coss": 1.8e9,
                "l_coss": 2.5e-3,
                "Damping": 1.0e6,
                "A": 1.0e-24,
                "PhiViscLim": 0.2,
                "n": 3.0,
                "sy": 9.5e6,
                "hardening": -0.9,
                "hard_eRef": 2.0e-4,
                "pl_visc": 1.0e3
            },
```

which defines:

- The rheology used for the material, allowing for the following options: “LinearElastic” (only elastic deformations), “ViscoElastic” (elastic and viscous deformations), “ViscoElasticExpl” (elastic and viscous deformations, with an explicit time integration scheme for the viscous creep), “DamViscoElastic” (elastic and viscous deformations, with damage affecting the viscous driving force), “DamViscoElasticExpl” (elastic and viscous deformations, with damage affecting the viscous driving force, and an explicit time integration scheme), “ViscoElasticPlastic” (elastic, viscous, and plastic deformations), and “DamViscoElasticPlastic” (elastic, viscous, and plastic deformations, with damage affecting the viscous driving force).
- The density, Young’s modulus and Poisson’s ratio of the solid material.
- The Cosserat shear modulus and length scale.
- Damping coefficient used for the additional damping.
- Glen’s law creep coefficient, exponent, and an optional limit beyond which no more creep occurs (for stability).
- The yield strength, and hardening parameters, and viscous regularization used within the Von Mises plasticity model.

The fracture-specific properties:

TestCases/TriAxialCompression/TriAxial.json

```
            "FractureProperties":{
                "Split": "SpecStress",
                "l": 1.5e-3,
                "Gc": 4000.0e-3,
                "Viscosity": 1.0e-1
            },
```

defining the specific phase-field energy split used (using "specStress" for the results used in the paper), phase field legnth scale, fracture energy release rate, and a viscous regularisation parameter. Finally, defining the thermal properties:

TestCases/TriAxialCompression/TriAxial.json

```
            "ThermalProperties":{
                "ThermalConductivity": 2.0,
                "HeatCapacity": 2.1e3
            }
```

1.3.4. Solver settings

For the linear and non-linear solver, we define the degrees of freedom that require solving for, and the staggered solver step they are solved within:

TestCases/TriAxialCompression/TriAxial.json

```
    "Dofs":{
        "DofNames": ["ux","uy","uz", "ox", "oy", "oz","phase","lMult_pf","lMult_pf2","T"],
        "DofStep":  [0,   0,    0,      0,    0,    0,      1,      1,  1,    2]
    },
```

Additionally, we define a block with the solver parameters:

TestCases/TriAxialCompression/TriAxial.json

```
    "nonlinSolver":{
        "max_it": 40,
        "max_outer_it": 1,
        "convCrit": [1.0e-5, 1.0e10, 1.0e-5],
        "tiny": [1.0e-5, 1.0e10, 1.0e-8],
        "linesearch": true,
        "linesearchLims": [0.1, 1.0],
        "LinSolver":{
            "Type": "GMRES",
            "IOptionIdx": [],
            "IOptionVals": []
        },
```

Defining the maximum amount of Newton-Raphson iterations per staggered step, the amount of multi-pass iterations for the staggered solution scheme (1 for a single pass staggered scheme, 2 or more for a multi-pass staggered scheme), and the convergence criteria for the non-linear solver (displacement, force, and energy based thresholds). This also defines whether a line-search algorithm is used, and the limits for the line-search step size. Finally, the type of linear solver is defined, with "GMRES" combining a GMRES iterative solver with a schwarz preconditioner (solving the linear system on each core separately), and "MUMPS" and "PARDISO" using direct LU solver.

1.3.5. Initialisation settings

Optionally, an initial state can be prescribed for selected test cases:

TestCases/TriAxialCompression/TriAxial.json

```
        "Initialization":{
            "Type": "Defect",
```

```
            "InteriorGroup": "internal",
            "Material": "Ice",
            "Z":150e-3,
            "Y":50e-3,
            "R":0.0,
            "T0": -11.0
        }
```

which defines the type of initialisation, the interior element group and material it applies to, and the geometric and thermal parameters used for the initial defect field.

1.3.6. Time integration settings

Time integration parameters are defined through:

TestCases/TriAxialCompression/TriAxial.json

```
        "TimeDiscretisation":{
            "Scheme": "BDF2",
            "_Scheme": "Newmark",
            "beta": 0.4,
            "gamma": 0.75
        }
```

for selecting the time integration scheme used by the momentum equation (e.g. BDF2 or Newmark), together with the associated Newmark parameters when this scheme is selected.

The simulation timeline and adaptive time stepping settings are then defined as:

TestCases/TriAxialCompression/TriAxial.json

```
    "TimeSolver":{
        "dtSteps": [1.0, 0.025],
        "tSteps": [-100, 0],
        "DynamicDT": true,
        "TimeDataRefine": "PhaseField1/dL_crack",
        "TimeDataRefineFactor": 5.0e-4,
        "ScaleMin":1e-4,
        "t0": -10.0,
        "tmax": 60.0,
        "outputN": 10
    },
```

This defines piece-wise constant time step sizes (*dtSteps*) over user-defined intervals (*tSteps*), whether dynamic time-step reduction is active, which output quantity is monitored for refinement, the refinement scaling and lower bound, the start and end time of the analysis, and how often output is written. Specifically for the time increment refinement, at the end of each time increment the monitored quantity is compared to the value set in the input file, and if it is larger, the time increment is reduced and the time step is repeated.

1.3.7. Visualisation settings

For live visualisation during the run, plots are selected and configured through:

TestCases/TriAxialCompression/TriAxial.json

```
    "Visualisation":{
        "_Plots": ["uz","T","PhaseField","PhaseField2","plot","plot2","plot3"],
        "Plots": ["PhaseField","plot","plot3"],
        "ElementGroup_u": "side",
```

where *Plots* defines which visualisations are shown, and each named entry specifies the settings for a particular surface, contour, or XY plot. For example:

TestCases/TriAxialCompression/TriAxial.json

```
        "plot":{
            "Type": "Plot_XY",
```

```
            "x": "Tri/U_Ext",
            "y": ["Tri/p_Ext"],
            "Frequency": "step"
        },
        "plot2":{
            "Type": "Plot_XY",
            "x": "time",
            "y": ["PhaseField1/L_crack"],
            "Frequency": "step",
            "xlim": "0+"
        },
        "plot3":{
            "Type": "Plot_XY",
            "x": "tstep",
            "y": ["dt"],
            "Frequency": "step",
            "xlim": "0+",
            "ylim": "log"
        }
```

which defines XY plots by selecting the x-axis quantity, one or more y-axis quantities, the update frequency, and optional axis limits.

1.3.8. Output settings

The output file settings are defined by:

TestCases/TriAxialCompression/TriAxial.json

```
    "Outputs":{
        "OutputMeshOnlyOnce": true,
        "SaveFolder":"Results",
        "ElementGroups": ["internal"],
        "internal":{
            "Nodes": ["ux","uy","uz","phase","T"],
            "IP":["s1","s2","s3","dam"]
        }
    },
```

which controls whether the mesh is written once or every output step, the destination folder, and which element groups and fields are exported (both nodal and integration point quantities).

1.3.9. Model definitions

Finally, the active models are listed and configured through:

TestCases/TriAxialCompression/TriAxial.json

```
    "Models":{
        "Names":["PhaseField1","Thermal","Boundary1","Boundary2","Tri"],
        "PhaseField1":{
            "Name": "Fracture/CossPhaseField",
```

The *Names* list defines all the models being used, and is then followed by a model-specific block defining the domain the model is active on, and any related settings. The required settings will be defined within the next section.

2. Physics Models

The code includes several physics models, which are activated through the input file. This section describes the main models implemented, and the required settings. For a more complete overview of the governing equations, please refer to the paper.

2.1. Cosserat phase field model

This model solves the phase-field and momentum balance equations on the interior of the domain:

$$\boldsymbol{L}_{\mathrm{c}}^{T}\left(d_{\mathrm{e}}\sigma^{+}+\sigma^{-}\right)-d_{\mathrm{i}}\boldsymbol{\rho}_{\mathrm{c}}\ddot{\mathbf{u}}_{\mathrm{c}}+d_{\mathrm{i}}\boldsymbol{C}_{\mathrm{c}}\dot{\mathbf{u}}_{\mathrm{c}}+d_{\mathrm{g}}\mathbf{b}=0 \tag{1}$$

$$\begin{aligned}&\frac{G_{\mathrm{c}}}{4\ell}\left(2-\ell^{2}\nabla^{2}\phi-\frac{1}{8}\ell^{4}\nabla^{4}\phi\right)+\zeta\dot{\phi}+\frac{\partial\mathcal{L}(\phi,L_{1},L_{2})}{\partial\phi}\\&\quad=-\frac{\partial d_{\mathrm{e}}}{\partial\phi}\Psi^{+}-\frac{\partial d_{\mathrm{i}}}{\partial\phi}\frac{\dot{\mathbf{u}}_{\mathrm{c}}^{T}\boldsymbol{\rho}_{\mathrm{c}}\dot{\mathbf{u}}_{\mathrm{c}}}{2}-\frac{\partial d_{\mathrm{g}}}{\partial\phi}\mathbf{b}^{T}\mathbf{u}_{\mathrm{c}}-\int_{t}\frac{\partial\beta_{\mathrm{p}}}{\partial\phi}\dot{\Psi}_{\mathrm{p}}+\frac{\partial\beta_{\mathrm{v}}}{\partial\phi}\dot{\Psi}_{\mathrm{v}}+\frac{1}{2}\frac{\partial d_{\mathrm{i}}}{\partial\phi}\dot{\mathbf{u}}\boldsymbol{C}_{\mathrm{c}}\dot{\mathbf{u}}\,\mathrm{d}t\end{aligned} \tag{2}$$

(here using a higher-order linear phase-field model). This model requires the following settings to be defined din the model-specific settings block:

TestCases/TriAxialCompression/TriAxial.json

```
        "PhaseField1":{
            "Name": "Fracture/CossPhaseField",
            "ElementGroup_u": "internal",
            "ElementGroup_phase": "internal",
            "ElementGroup_T": "internal",
            "Material": "Ice",
            "ViscousStab": 0.0e1,
            "Irreversible": "AugmentedLagrangeMult",
            "Intertia": true,
            "Gravity": false,
            "LumpedPFCapacity": false,
            "LumpedDriving": false,
            "ViscousHeating": true
        },
```

where the elementgroup options refer to the groups used to define the interior (these can represent different element orders, as long as they cover the same area/volume), "Material" refers to the name used to define the material properties, "Irreversible" defines whether to use an augmented lagrangian approach to enforce irreversibility of the phase-field or to use a history-based approach. The "Inertia" and "Gravity" toggles allow these to be turned on/off, and the "ViscousHeating" defines whether to include the coupling between viscous dissipation and temperature changes.

2.2. Thermal model

The thermal model defines the temperature, and implements the temperature evolution as:

$$\rho c_{\mathrm{p}}\dot{T}-\nabla\cdot(k\nabla T)-Q_{\mathrm{melt}}=\dot{\Psi}_{\mathrm{p}}+\dot{\Psi}_{\mathrm{v}}+\dot{\Psi}_{\mathrm{f}}+\dot{\Psi}_{\mathrm{pd}}+\dot{\Psi}_{\mathrm{d}} \tag{3}$$

where Q_{melt} is only present if the variable porosity model is active, and the right-hand terms depend on the used solid model. Input parameters are given by:

TestCases/TriAxialCompression/TriAxial.json

```
        "Thermal":{
            "Name": "Thermal/ThermalModel",
            "ElementGroup_T": "internal",
            "Material": "Ice"
        },
```

2.3. Variable porosity model

This model defines the porosity (otherwise, it is assumed to be constant), and allows it to evolve based on melting and re-freezing based on the thermal balance:

$$\rho\Lambda_{\mathrm{m}}\dot{\psi}+Q_{\mathrm{melt}}=0 \tag{4}$$

with the input parameters defined as:

TestCases/IceCliffs/IceCliff.json

```
        "Porosity":{
            "Name": "PoroElasticity/VariablePorosity",
            "ElementGroup_T": "internal",
            "ElementGroup_poros": "internal",
            "Material": "Ice",
            "Fluid": "Water",
            "PhaseChange": false
        },
```

which requires defining both the material parameters to use for the solid and fluid materials, and whether to allow for phase-changes (melting and re-freezing) or to just define the porosity degree of freedom, but keep it constant.

2.4. Boundary constraints model

The boundary constraints models implement fixed (Dirichlet) boundary conditions, e.g. imposing fixed displacements on a defined boundary. These models are defined by setting the following parameters:

TestCases/TriAxialCompression/TriAxial.json

```
        "Boundary2":{
            "Name": "Constraints",
            "NodeGroup":"bottom",
            "dofs": ["ux","uy","uz","ox","oy","oz"],
            "values": [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
        },
```

Where the node group defines the boundary (or interior) nodes that are being constrained, the list of dofs defines the degrees of freedom, and the values lists the value to which these degree of freedoms are being constrained. For example, to constrain the x-displacement to zero on a boundary, the dofs would be ["ux"], and the values would be [0.0]. Note that these boundary conditions are set once, and then kept constant throughout the simulation. Implementation-wise, these boundary conditions are implemented by eliminating them from the residual force vector, and tangent matrix.

2.5. Tri-Axial compression forcing model

This model is specific to the tri-axial compression case, where it implements the forcing applied to the exterior boundaries and the imposed displacement at the top surface:

TestCases/TriAxialCompression/TriAxial.json

```
        "Tri":{
            "Name": "TriAxial",
            "SideGroup":"side",
            "TopGroup":"top",
            "p0": 0.0e6,
            "dU": -1e-4,
            "DamagedForces": true,
            "dummy": 1.0e14,
            "Material": "Ice",
            "t_postFail": 5e-1
        }
```

within this model, the imposed hydrostatic pressure is defined by "p0", the loading rate by "dU", and the penalty stiffness used to impose the displacement rate by "dummy". This model also includes the option to end simulations based on when failure occurs (defined as dropping below 5% of the peak load) by setting the post-failure time. Aditionally, the imposed displacement, and resulting forcing are saved to the output files through this model.

2.6. Weak external forcing model

Finally, the weak external force model imposes boundary conditions through a penalty approach, allowing them to be turned on/off after a certain time. This model is used by defining:

TestCases/IceCliffs/IceCliff.json

```
        "Boundary3":{
            "Name": "WeakExternal",
            "ElementGroup":"right",
            "k": 1.0e12,
            "dofs": ["ux"],
            "values": [0.0],
            "UpperTime": 0.0,
            "Lumped": true
        }
```

The contributions to the force vector resulting from this model are either integrated using a standard Gauss integration scheme, or using a Lumped scheme, where the forces are directly allocated to nodes.

3. Examples

Several example files are included in the *TestCases* folder, including 2D and 3D tri-axial compression cases, and 2D ice-cliff failure. Each of these examples can be run (after generating the required mesh) by running the command *./IceCode <inputfile>* from the base directory, and visualised using the included *PlotData.m* matlab script.

3.1. Mesh Generation

Before running the simulations, the required meshes need to be generated using the MATLAB script in the MeshGen folder. For example, for the 2D ice-cliff case, running the *TestCases/IceCliffs/MeshGen/generate_tmesh.m*. This file has several parameters that can be adjusted:

TestCases/IceCliffs/MeshGen/generate_tmesh.m

```
cx = 14;
cy = 4;
for rat=[5]
    for dx=[1]
        for H = [200]
```

where the values for *cx* and *cy* are used to define the distribution of the mesh across the cpu cores (creating a mesh distributed for cx $\times$ cy cores). *dx* sets the element size, and *H* and *rat* define the height of the cliff, and ratio between the height and length of the domain.

3.2. Results while solving

Once the mesh is generated, run the cliff test case using *mpirun -np 56 ./IceCode TestCases/IceCliffs/IceCliff.json* from the base directory (adapt the number of processes to the amount available, and the generated mesh). During the run, several windows will open, Fig. 1, showing the state of the system, including surface plots of the phase-field variable, displacements, and temperature.

3.3. Post-processing

While running the simulation, outputs are saved to the *Results* folder, creating:

- Backup.hdf5: This file is used for restarting simulations
- DofSpace.hdf5: Only used for debugging purposes.
- mesh_0.hdf5: This file contains the mesh information in an easy-to-post-process format. Specifically, it contains the corners of each element and locations of each integration point for which data is saved.

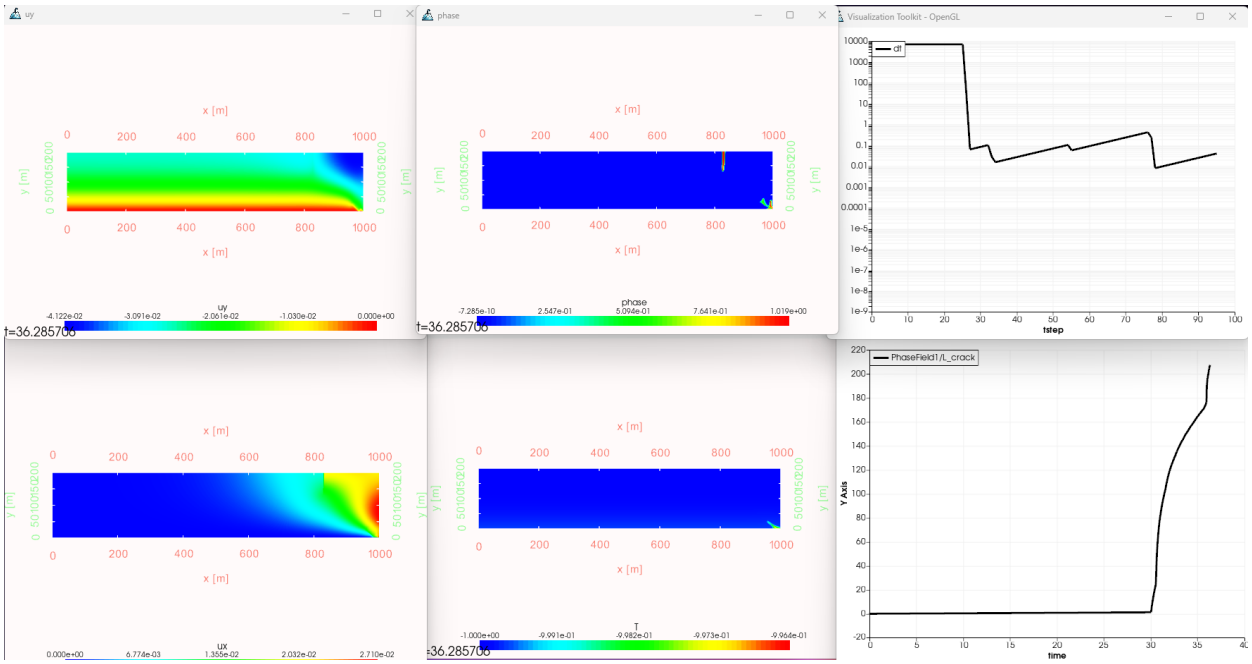

Figure 1: Example of figures that open while running the simulation (top left to bottom right): Vertical displacements, phase-field, time increment size vs time increment, horizontal displacement, temperature, and estimated total crack length vs time.

- results_n.hdf5: These files contain the outputs from the simulation, with n being the output step for which they are saved. Each field is saved can be accessed through its location and name, e.g. accessing the temperature as "internal/T". This file also contains the field "time", indicating to which simulation time the saved results correspond.
- TimeData.hdf5: This file contains the global measures: The time, time increment size, crack length, and crack length change. For tri-axial simulations, it also contains the applied displacement and resulting force.

These files use the standard HDF file format [10], and therefore can be easily read using a variety of software, including python and MATLAB. Post-processing scripts using MATLAB are included in the test cases' folders, which process the simulations results to animations and figures. For example, running the *TestCases/IceCliffs/PlotData.m* script will create the figures shown in Fig. 2.

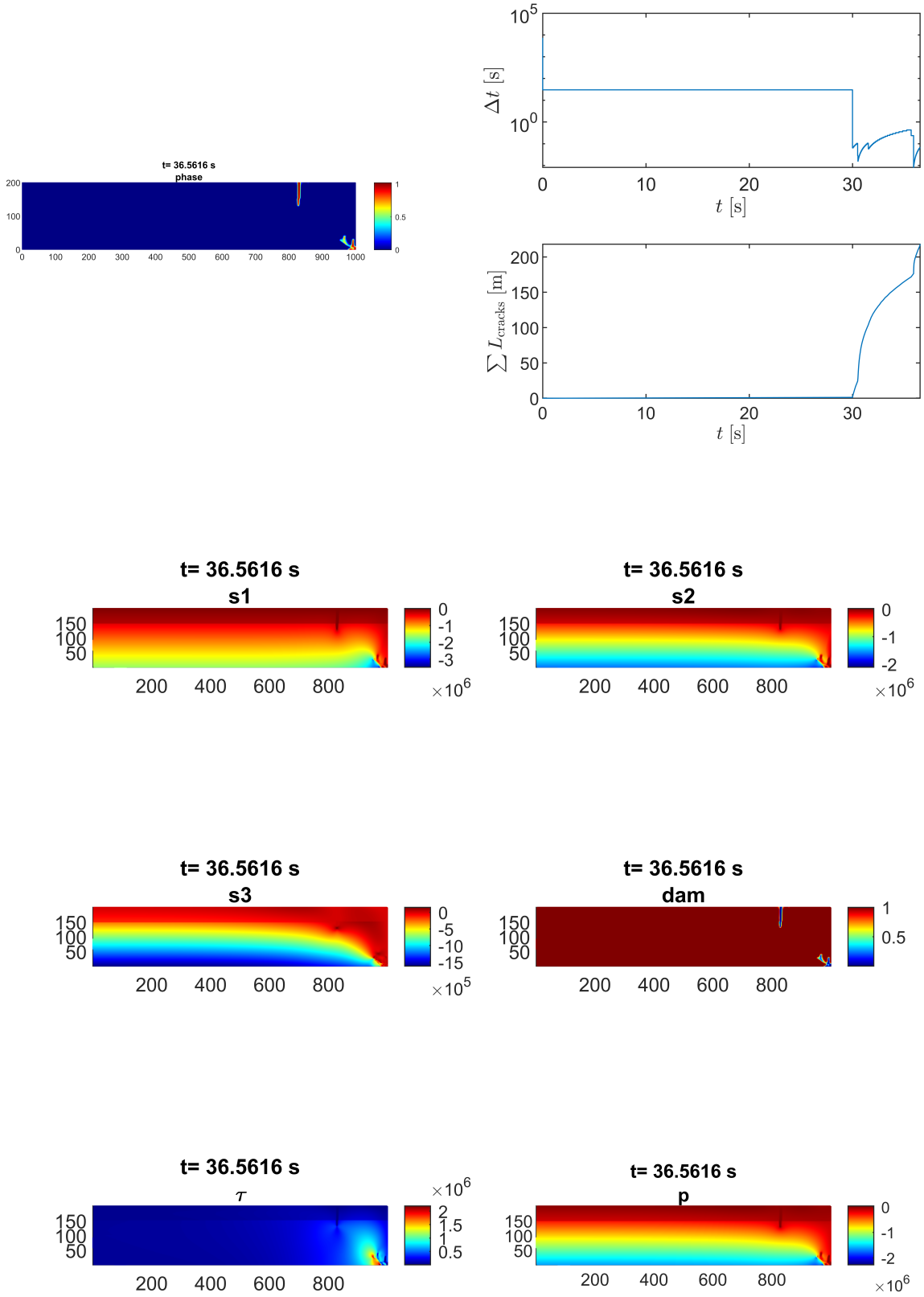

Figure 2: Post-processed results from the ice-cliff failure simulation, showing the phase-field variable at the final time step (top left), the estimated total crack length vs time (top right), and the vertical and horizontal stress at the final time step (bottom).

4. References

[1] Microsoft, Windows Subsystem for Linux Documentation, https://learn.microsoft.com/en-us/windows/wsl/.

[2] Accelerate Fast Math with Intel® oneAPI Math Kernel Library, https://www.intel.com/content/www/us/en/developer/tools/oneapi/onem

[3] Libeigen / eigen · GitLab, https://gitlab.com/libeigen/eigen (Apr. 2025).

[4] Tencent/rapidjson, Tencent (Apr. 2025).

[5] W. J. Schroeder, K. Martin, W. E. Lorensen, L. S. Avila, K. W. Martin, B. Lorensen, The Visualization Toolkit: An Object-Oriented Approach to 3D Graphics ; [Visualize Data in 3D - Medical, Engineering or Scientific ; Build Your Own Applications with C++, Tcl, Java or Python ; Includes Source Code for VTK (Supports UNIX, Windows and Mac)], 4th Edition, Kitware, Inc, Clifton Park, NY, 2006.

[6] A. Devresse, N. Cornu, L. Grosheintz-Laval, O. Awile, T. de Geus, F. Pereira, M. Wolf, HighFive Contributors, HighFive - Header-only C++ HDF5 interface, Zenodo (Dec. 2024). doi:10.5281/ZENODO.10679422.

[7] S. Balay, S. Abhyankar, M. Adams, S. Benson, J. Brown, P. Brune, K. Buschelman, E. Constantinescu, L. Dalcin, A. Dener, V. Eijkhout, J. Faibussowitsch, W. Gropp, V. Hapla, T. Isaac, P. Jolivet, D. Karpeev, D. Kaushik, M. Knepley, F. Kong, S. Kruger, D. May, L. McInnes, R. Mills, L. Mitchell, T. Munson, J. Roman, K. Rupp, P. Sanan, J. Sarich, B. Smith, S. Zampini, H. Zhang, J. Zhang, PETSc/TAO Users Manual (Rev. 3.20), Tech. Rep. ANL–21/39 Rev. 3-20, 2205494, 185712 (Nov. 2023). doi:10.2172/2205494.

[8] V. P. Nguyen, C. Anitescu, S. P. A. Bordas, T. Rabczuk, Isogeometric analysis: An overview and computer implementation aspects, Mathematics and Computers in Simulation 117 (2015) 89–116. doi:10.1016/j.matcom.2015.05.008.
URL https://www.sciencedirect.com/science/article/pii/S0378475415001214

[9] Igafem.
URL https://sourceforge.net/projects/cmcodes/

[10] H. Group, HDF5.
URL https://www.hdfgroup.org/HDF5/